

Major Assignment: Building a Mini Transformer and Producing a Small Benchmark

1. Introduction

This report states the details of the implementation and evaluation of a Mini Transformer encoder built from scratch using PyTorch. The objective of this model was to solve a synthetic sequence classification task to determine whether the first valid token in a padded sequence appears again in the second half of that sequence. This problem requires the model to identify specific token positions and draw connections across various sequence lengths. Therefore, it is an ideal test for self-attention mechanisms and positional encodings.

2. Methodology

The model architecture was implemented without using pre-built PyTorch Transformer modules (`torch.nn.Transformer`). It strictly follows the standard encoder blueprint as follows

2.1 Token & Positional Embeddings

I used standard `nn.Embedding` to map discrete vocabulary tokens into dense vectors of dimension $d_{\text{model}} = 64$. These embeddings were then combined with sinusoidal positional encoding to inject sequential order. The sinusoidal positional encodings use the following formula to ensure that the model can mathematically distinguish the absolute and relative positions of tokens.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

Where pos is the position of the token in the sequence and i is the index. The token embeddings were scaled by square root of d_{model} before adding the positional encodings. This is to maintain stability throughout the forward pass.

2.2 Multi-Head Self-Attention

The core attention mechanism was implemented manually to allow the model to draw contextual relationships between the first token and the second half of the sequence. The input dimension was projected into Queries (Q), Keys (K), and Values (V). There were then split across 4 attention heads to allow the model to attend to different representational subspaces at the same time.

The scaled dot-product attention for each head was computed as

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

A padding mask (M) was also applied before the softmax function. For any position corresponding to a PAD token, the mask applied a value approaching negative infinity ($-\infty$) to ensure the softmax output for those positions result in 0.0. This will prevent the model from attending to meaningless padding tokens. The outputs of all heads were then concatenated and passed through a final linear projection layer.

2.3 Position-wise Feed-Forward Network (FFN)

The output of the attention mechanism was processed by a two-layer feed-forward network applied independently to each position. This network introduces non-linearity to the model, allowing it to learn more complex decision boundaries. The FFN expands the 64-dimensional input to 128 dimensions, applies a ReLU activation function, and projects it back to 64 dimensions

$$FFN(x) = \max(0, xW_1 + b_1) W_2 + b_2$$

Dropout ($p = 0.1$) was applied after the activation function to avoid overfitting.

2.4 Transformer Block (Add & Norm)

The Multi-Head Self-Attention and FFN modules were wrapped inside a standard Transformer Encoder block. To solve the vanishing gradient problem and ensure training stability, residual connections and Layer Normalization were applied at each step. The flow for each sub-layer can be defined as

$$Output = LayerNorm(x + SubLayer(x))$$

2.5 Mean Pooling and Classification Head

As the assignment's task requires a single binary classification for the entire input sequence, the output of the final Transform block could not be used directly. It consists of a vector of very token. Therefore, instead of using a [CLS] token, the model aggregates the sequence via mean pooling.

The pooling operation explicitly masks out PAD tokens so they do not skew the average. The remaining valid token vectors are summed and divided by the total number of valid tokens in that specific sequence. This single, summary vector is then passed through a final linear layer to output a raw logit for binary classification.

2.6 Training Configuration

The model variants were trained for 10 epochs using a batch size of 32. The Adam optimizer was utilized with a learning rate of 0.001. Because the model outputs a raw logit for a binary classification task, Binary Cross Entropy with Logits Loss (BCEWithLogitsLoss) was selected as the criterion. It natively combines a Sigmoid layer and the BCELoss function for greater numerical stability.

3. Experimental Setup

To benchmark the architecture and isolate the impact of its core components, the training pipeline was standardized across all runs.

- **Vocabulary Size:** 5
- **Max Sequence Length:** 20
- **Embedding Dimension (d_model):** 64
- **Feed-Forward Dimension (d_ff):** 128
- **Dropout Rate:** 0.1
- **Batch Size:** 32
- **Epochs:** 10
- **Optimizer:** Adam
- **Learning Rate:** 0.001
- **Loss Function:** BCEWithLogitsLoss
- **Random Seed:** 67

Four distinct configurations were trained to isolate the effects of positional encodings, attention heads, and network depth.

1. **Base Model:** 1 Layer, 4 Attention Heads, Positional Encodings Enabled.
2. **No Positional Encoding (No PE):** 1 Layer, 4 Attention Heads, Positional Encodings Disabled.
3. **Single Head:** 1 Layer, 1 Attention Head, Positional Encodings Enabled.
4. **Deep Model:** 2 Layers, 4 Attention Heads, Positional Encodings Enabled.

4. Results and Discussion

The benchmark results obtained from the experiments are summarized in the table below

Configuration	Positional Encoding	Heads	Layers	Params	Val Acc (%)	Test Acc (%)	Train Time
Base	Yes	4	1	33,857	84.6	84.6	0.23
No PE	No	4	1	33,857	81.4	81.4	0.23
Single Head	Yes	1	1	33,857	85.0	85.0	0.22
Deep Model	Yes	4	2	67,265	86.1	86.1	0.38

4.1 Important of Positional Encoding

The most critical finding is the complete failure of the "No PE" model, which flatlined at 81.4% accuracy. Transformers process sequences simultaneously. Without positional encodings, they operate as a "bag of words." Because this task explicitly requires finding the "first" token and comparing it to the "second half," the model mathematically cannot solve the problem without positional timestamps.

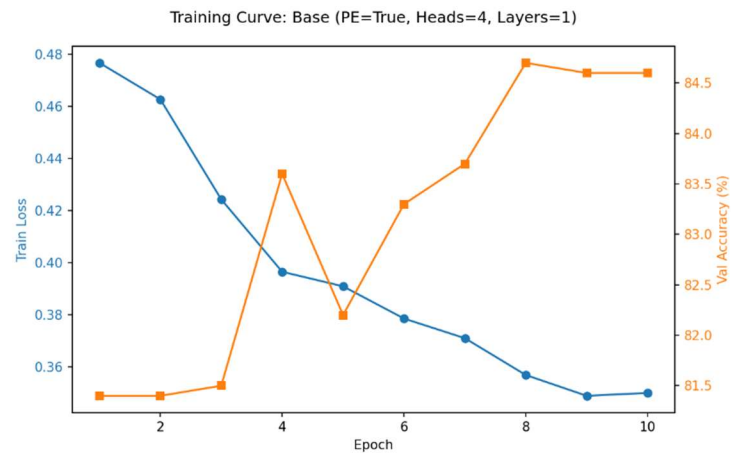


Figure 1: Training curve for the Base Model

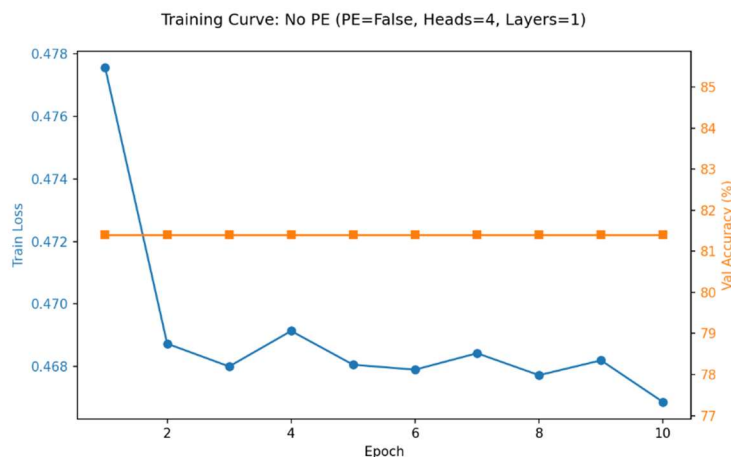


Figure 2: Training curve for the No PE Model

4.2 Single Head and Multi Head

While Multi-Head Attention excels in complex NLP by tracking different representational subspaces such as grammar and tones, this synthetic task relies on a single heuristic. The Single Head model (85.0%) performed just as well as the 4-Head Base model (84.6%). For simplistic logic, one attention head is fully capable, and adding more can introduce unnecessary noise.

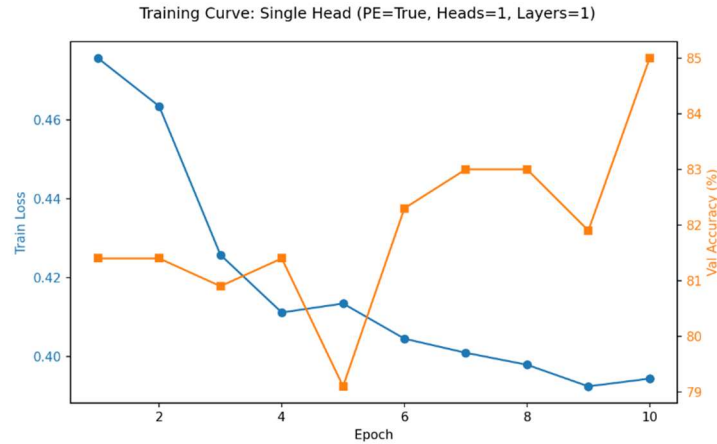


Figure 3: Training curve for the Single Head Model

4.3 Diminishing Returns on Depth

The Two-Layer model achieved the highest accuracy (86.1%) but required doubling the parameter count. While the extra depth allows for smoother feature extraction, the baseline 1-layer architecture is already highly efficient for this assignment's problem, making deeper networks redundant.

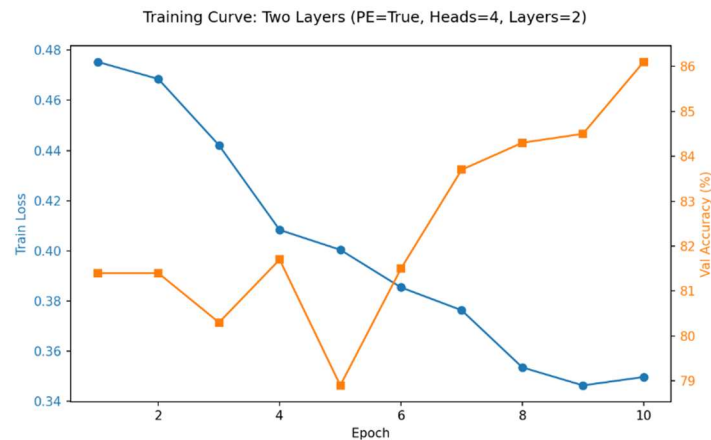


Figure 4: Training curve for the Two Layer Model

5. Conclusion

The Mini Transformer successfully learned the sequence task. The benchmarks prove that positional encodings are an absolute necessity for order-dependent logic. Furthermore, it demonstrated that while multi-head and deep architectures dominate large language models, lightweight configurations such as single head and single layer are highly optimal when the underlying task rule is straightforward.